인공지능기초 · 6주차 실습 · 25분

가중치는 어떻게 바뀔까?

코드 한 줄을 수식 한 줄로 읽는 역전파 실습

```
grad = dL_dyhat * dyhat_dh * dh_dz * dz_dw
```

실습 지도

오늘 실행할 9개 셀과 확인할 숫자

셀	코드가 하는 일	화면에서 확인할 숫자	시간
1	sigmoid 정의, 출력 자릿수 설정	NumPy 버전	2–3분
2	뉴런 하나의 순전파	loss = 0.071268	3–5분
3	연쇄법칙 기울기 + 수치 미분 검산	grad = −0.088723	5–8분
4	가중치 한 번 수정	w: 0.5000 → 0.5444	8–10분
5	XOR 데이터, 매개변수 초기화	W1 (2, 4), W2 (4, 1)	11–12분
6	행렬 순전파, 교차 엔트로피 손실	초기 손실 0.6994	12–15분
7	역전파 함수, 5000번 반복 학습	최종 손실 0.0019	15–19분
8	학습 곡선, 결정 영역 그림	확률 0.5 경계선	19–21분
9	학습률만 바꿔 비교	lr 0.01 / 1.0 / 200	21–24분

색 띠 셀 2–4: 숫자 하나로 계산 · 셀 5–9: 같은 계산을 행렬로

준비 코드에서 눈여겨볼 두 줄

```
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(precision=4, suppress=True)
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
```

set_printoptions

결과를 소수 4자리로 보여 줍니다. 화면 숫자와 슬라이드 숫자를 바로 비교할 수 있습니다.

np.clip

z가 아주 크면 exp가 넘칩니다. 범위를 잘라도 시그모이드 값은 사실상 그대로입니다.

@ 기호

행렬 곱입니다. 셀 6부터 등장합니다.

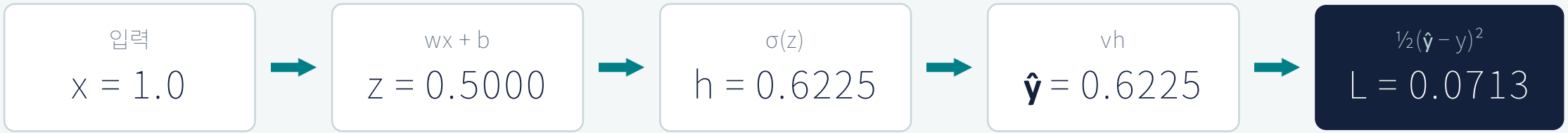
코드 네 줄이 수식 네 개와 한 줄씩 대응한다

```
x, y = 1.0, 1.0
w, b, v = 0.5, 0.0, 1.0
z = w*x + b
h = sigmoid(z)
y_hat = v*h
loss = 0.5*(y_hat - y)**2
```

회색 두 줄은 입력, 정답, 매개변수의 시작값입니다.

출력층은 선형($\hat{y} = vh$)으로 두어 계산을 단순하게 했습니다.

이 예에서는 w만 수정하고 b와 v는 고정합니다.



변수 이름 dA_dB는 ∂A/∂B로 읽는다

코드 변수	수식	값	의미
dL_dyhat	$\partial L/\partial \hat{y} = \hat{y} - y$	−0.3775	예측이 정답보다 작다
dyhat_dh	$\partial \hat{y}/\partial h = v$	1.0	출력층 가중치
dh_dz	$\partial h/\partial z = h(1-h)$	0.2350	시그모이드 미분 (최대 0.25)
dz_dw	$\partial z/\partial w = x$	1.0	입력

```
grad = dL_dyhat*dyhat_dh*dh_dz*dz_dw
     = −0.3775 × 1.0 × 0.2350 × 1.0 = −0.088723
```

w를 직접 조금 움직여 연쇄법칙을 채점한다

```
def scalar_loss(weight):
    return 0.5*(v*sigmoid(weight*x+b)-y)**2
eps =1e-5
numeric_grad =(scalar_loss(w+eps)-scalar_loss(w-eps))/(2*eps)
assert np.isclose(grad, numeric_grad, atol=1e-8)
```

scalar_loss

셀 2의 계산 전체를 w만 받는 함수로 묶은 것

중앙 차분

w를 ±0.00001 움직였을 때 손실 차이 ÷ 0.00002

assert

두 값이 다르면 셀이 멈춘다. 둘 다 −0.088723

음수 기울기를 빼면 w는 커진다

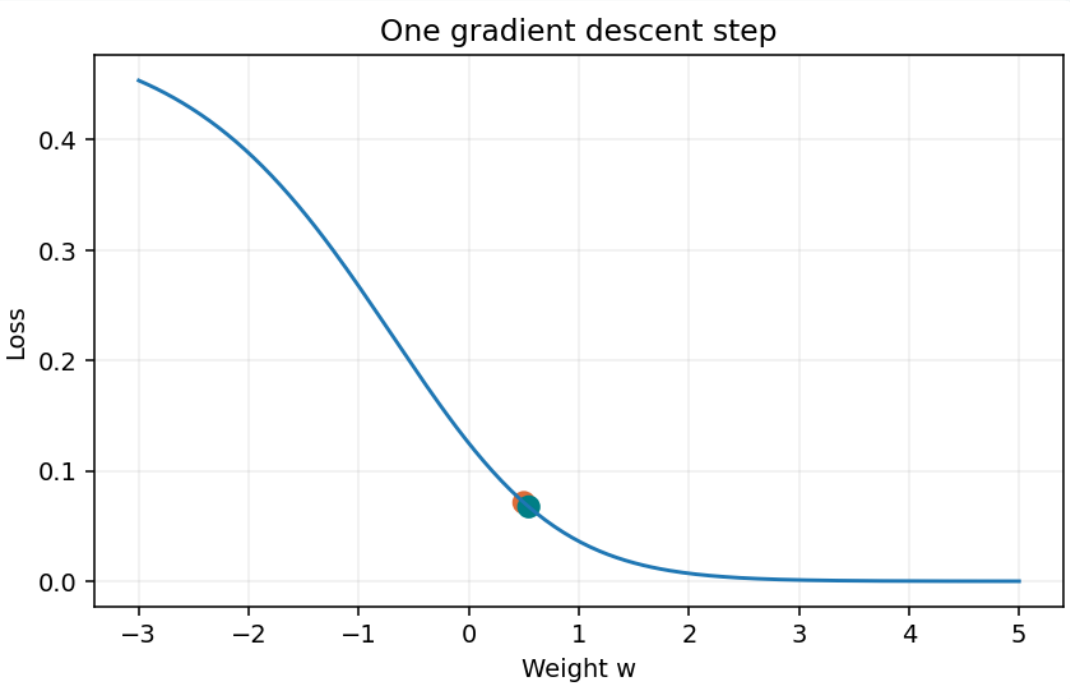
```
lr = 0.5
w_new = w - lr*grad
```

$0.5 - 0.5 \times (-0.0887)$

$= 0.5 + 0.0444 = 0.5444$

손실: 0.071268 → 0.067408

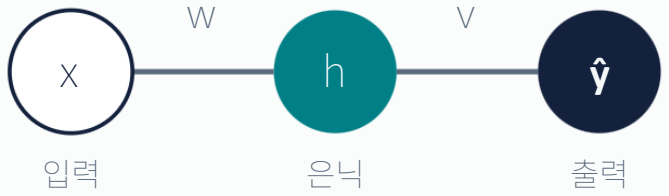
그래프: 주황 = 수정 전, 청록 = 수정 후



셀 2–4에서 셀 5–7로

숫자 하나가 행렬 하나로 바뀔 뿐, 계산 순서는 같다

셀 2–4 · 표본 1개, 뉴런 1개

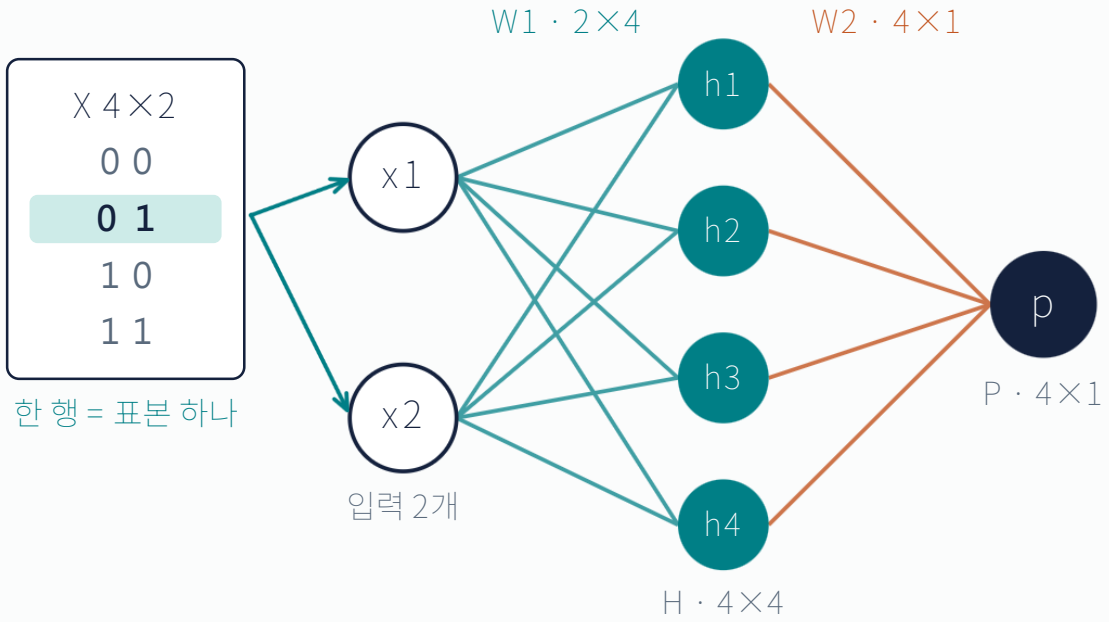


z = w*x + b

y_hat = v*h

확장

셀 5–7 · 표본 4개, 은닉 뉴런 4개



소문자 = 숫자 하나 · 대문자 = 행렬 · 행렬의 행 = 표본

손실: $\frac{1}{2}(\hat{y} - y)^2$ → 교차 엔트로피

매개변수는 이름표가 붙은 딕셔너리 하나에 담긴다

```
X = np.array([[0,0], [0,1], [1,0], [1,1]], dtype=float)
Y = np.array([[0], [1], [1], [0]], dtype=float)
def init_params(seed=7):
    rng = np.random.default_rng(seed)
    return {'W1': rng.normal(0, 1, (2,4)), 'b1': np.zeros((1,4)),
            'W2': rng.normal(0, 1, (4,1)), 'b2': np.zeros((1,1))}
```

X와 Y

행 4개 = 표본 4개. 입력이 다르면 1, 같으면 0

딕셔너리 p

p['W1']처럼 이름으로 꺼내고, 같은 이름으로 수정

seed=7

실행할 때마다 같은 초기값. 셀 9 비교의 전제

forward()는 행렬 크기를 따라가며 읽는다

```
def forward(X, p):
    H = sigmoid(X @ p['W1'] + p['b1'])
    Z2 = H @ p['W2'] + p['b2']
    P = sigmoid(Z2)
    return H, Z2, P
```

X 4×2 @ W1 2×4 + b1 1×4 → σ → H 4×4

H 4×4 @ W2 4×1 + b2 1×1 = Z2 4×1 → σ → P 4×1

곱하는 두 행렬의 안쪽 숫자가 같아야 곱할 수 있습니다 (2=2, 4=4). b1은 4개 행에 똑같이 더해집니다(브로드캐스팅).

손실은 P가 아니라 Z2로 계산한다

```
def bce(Z2, Y):
    return np.mean(np.logaddexp(0, Z2) - Y*Z2)
```

같은 식, 다른 계산

−[y log p + (1−y) log(1−p)]
= log(1 + e^z) − yz

왜 Z2인가

p가 0에 가까우면 log p가 −∞. logaddexp는 그 문제 없이 log(1 + e^z)를 계산한다.

학습 전 손실

0.6994

ln 2 = 0.6931에 가깝다. 네 표본 모두 확률을 0.5 근처로 주는 '모르는 상태'

backward()는 셀 3의 곱셈을 행렬로 한 것이다

연쇄법칙의 항	셀 3 (숫자 하나)	셀 7 backward()
출력의 오차	y_hat - y	dZ2 = (P-Y)/len(X)
뒤 가중치 곱하기	* v	dH = dZ2 @ p['W2'].T
시그모이드 미분	* h*(1-h)	dZ1 = dH*H*(1-H)
입력 곱하기	* x	X.T @ dZ1

출력 오차가 왜 (P − Y) / n 인가

시그모이드와 교차 엔트로피를 함께 미분하면 σ′ 항이 약분되어 오차만 남습니다. n(=4)으로 나누는 것은 손실이 4개 표본의 평균이기 때문입니다.

기울기의 크기는 그 매개변수의 크기와 같다

코드	계산	결과	같은 크기의 짝
dZ2 = (P-Y)/len(X)	원소별 계산	4×1	Z2
dW2 = H.T @ dZ2	(4×4) @ (4×1)	4×1	W2
db2 = dZ2.sum(axis=0)	4개 표본 합	1×1	b2
dH = dZ2 @ W2.T	(4×1) @ (1×4)	4×4	H
dZ1 = dH*H*(1-H)	원소별 곱	4×4	은닉층 입력
X.T @ dZ1	(2×4) @ (4×4)	2×4	W1
dZ1.sum(axis=0)	4개 표본 합	1×4	b1

.T는 크기를 맞추려고, sum(axis=0)은 4개 표본의 기울기를 합치려고 씁니다.

train()의 한 epoch은 네 단계다

```
def train(lr=1.0, epochs=5000, seed=7):
    p = init_params(seed)
    losses = []
    for epoch in range(epochs):
        H, Z2, P = forward(X, p)     # ①
        losses.append(float(bce(Z2, Y))) # ②
        grads = backward(X, Y, H, P, p)  # ③
        for name in p:               # ④
            p[name] -= lr*grads[name]
    _, Z2, P = forward(X, p)
    losses.append(float(bce(Z2, Y)))
    return p, np.array(losses), P
```

① 순전파 · ② 손실 기록 · ③ 역전파 · ④ 수정

③ 다음에 ④

기울기를 모두 구한 뒤 한꺼번에 수정

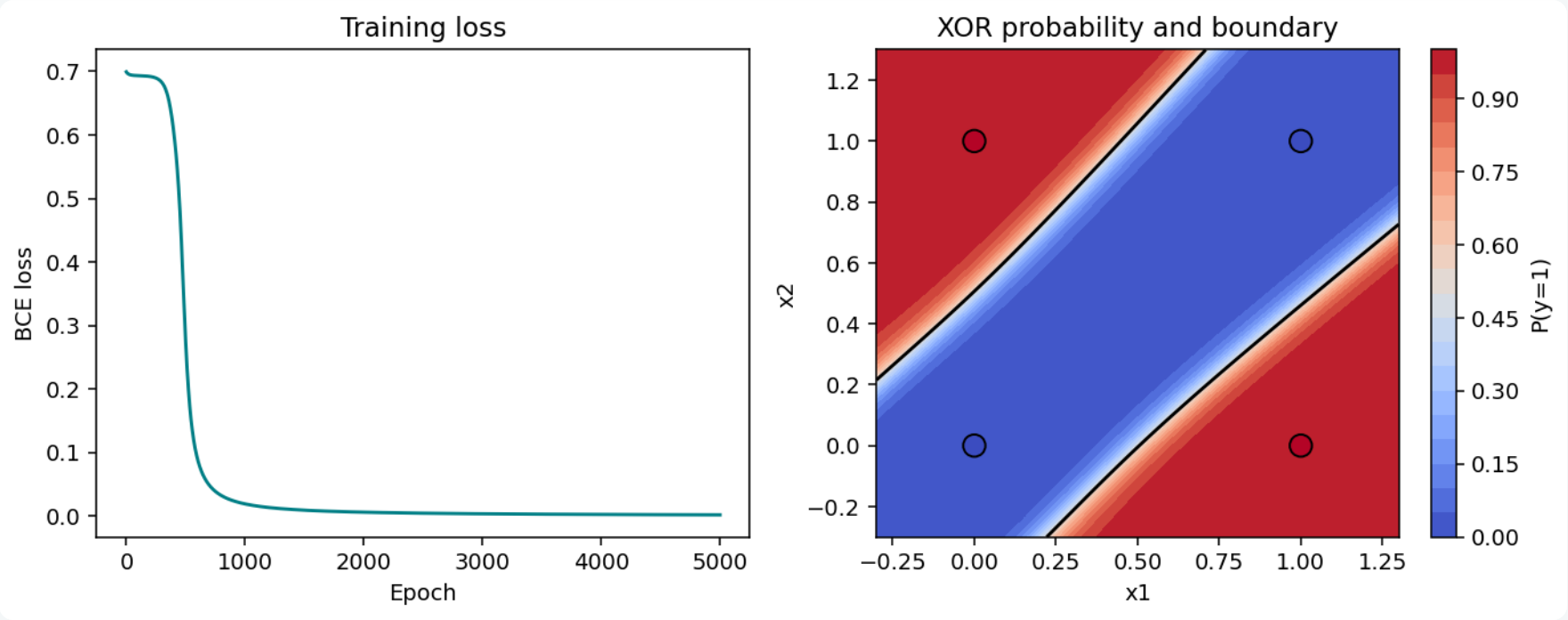
④ = 셀 4의 한 줄

w - lr*grad를 매개변수 4개에 반복

losses는 5001개

학습 전 1개 + 수정 5000번 후

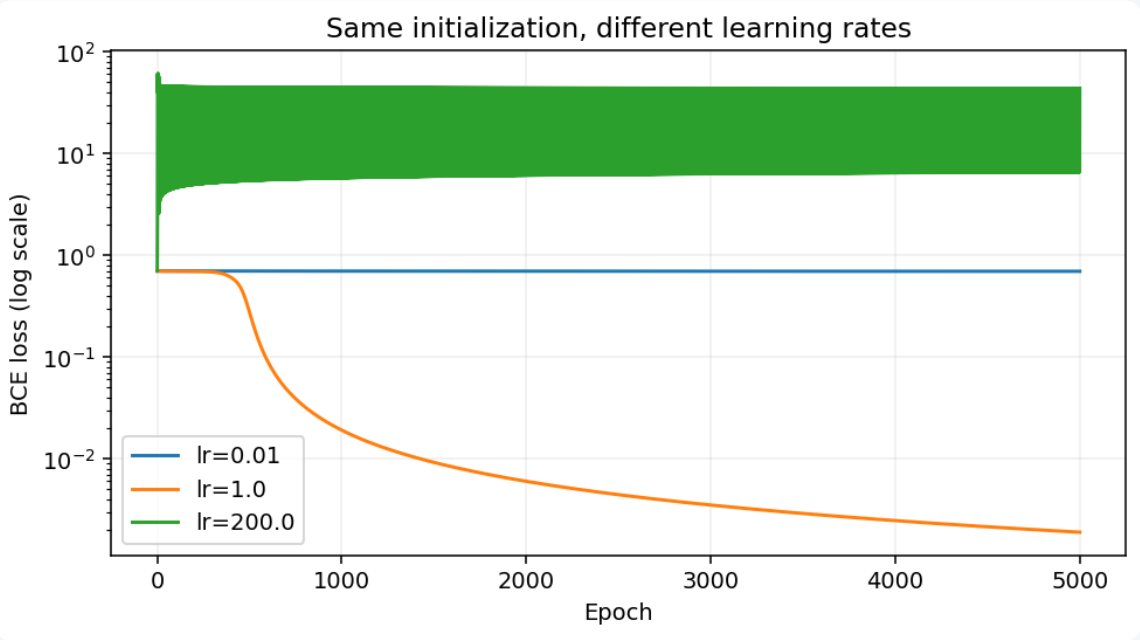
손실은 0.6994에서 0.0019로, 네 표본 모두 정답



학습 후 확률: (0,0) .0012 · (0,1) .9981 · (1,0) .9978 · (1,1) .0024

색 = 180×180 격자점에 forward()를 다시 적용한 확률

초기값은 고정하고 학습률 하나만 바꾼다




```
rates = [0.01, 1.0, 200.0]
train(lr=rate)  # seed=7
```

lr	최종 손실	정확도
0.01	0.694	50%
1.0	0.0019	100%
200	43.44	50%

0.01: 보폭이 작아 아직 ln 2 근처
200: 보폭이 커서 손실이 진동

확인 질문과 과제

코드를 보고 답해 보세요

질문 1 · 셀 4

grad가 음수일 때 w_new = w - lr*grad에서 w는 어느 쪽으로 움직이나?

질문 2 · 셀 7

backward()의 어느 줄이 셀 3의 dh_dz = h*(1-h)에 해당하나?

질문 3 · 셀 9

lr=0.01의 최종 손실이 ln 2 근처에 머문 이유는?

과제

셀 9의 rates만 [0.1, 0.5, 2.0]으로 바꿔 결과 표와 그래프를 제출하세요. 같은 초기값을 쓰는 이유와 가장 빨리 손실이 줄어든 설정을 2–3문장으로 설명하세요.